

SOMMERSEMESTER 2024

Studiengang/Abschluss: Wirtschaftsinformatik / B.Sc.

Prüfungsfach: Programmierung

Prüfungsnummer: **5706**

Prüfer: Prof. Dr. Andreas Biesdorf

Anzahl der Arbeitsblätter: 9 Seiten (ohne Deckblätter)

Bearbeitungszeit: 90 Minuten

Mit Bleistift oder in roter Farbe geschriebene Ausführungen werden nicht gewertet!

Zugelassene Hilfsmittel:

Das im Rahmen der Vorlesung erstellte Hilfsmittel.

Matrikelnummer:

--	--	--	--	--	--

Datum der Klausur:

..... **Platznummer:**

Punkte: _____ von **90** möglichen

Note: _____

Handzeichen des Prüfers: _____

Hinweise zu Klausuren im FB Wirtschaft

- Mit Aufnahme der Prüfung bringen Sie zum Ausdruck, dass Sie sich für prüfungstauglich halten. Ein Rücktritt von der Klausur während der Bearbeitungszeit ist damit nur noch in ganz besonderen Ausnahmefällen möglich.
- Sollten Sie zu Beginn der Bearbeitungszeit prüfungstauglich sein, dann aber während der Bearbeitungszeit prüfungsuntauglich werden, müssen Sie die Klausur abbrechen, Ihre Prüfungsuntauglichkeit den Aufsichtführenden anzeigen und schnellstmöglich, in jedem Fall aber fristgerecht für ein ärztliches Attest sorgen.
- Mit der Abgabe Ihrer Klausur bringen Sie zum Ausdruck, dass Sie sich während der gesamten Bearbeitungszeit für prüfungstauglich gehalten haben. Ein Rücktritt ist dann nicht mehr möglich.
- Jacken und Taschen sind in ausreichender Entfernung vom Platz zu deponieren.
- Die Bearbeitungszeit beginnt erst, wenn alle Klausuren ausgeteilt sind.
- Es sind nur die zugelassenen Hilfsmittel zu verwenden.
- Das Verlassen des Raumes für Toilettengänge während der Klausur ist nur nach Meldung und Bestätigung der Aufsicht gestattet. Die vorübergehende Abwesenheit wird im Protokoll vermerkt.
- Es sind keine anderen als die ausgegebenen Bearbeitungsblätter zu benutzen.
- Auf dem Tisch sind nur Schreibutensilien erlaubt, keine Mäppchen.
- Jeder Teilnehmer muss sich mit einem Studierendenausweis oder Lichtbildausweis ausweisen.
- Geräusche und Störungen jeder Art sind zu vermeiden.
- Die Klausurensätze sind grundsätzlich nicht zu öffnen. Wer den Klausurensatz öffnet, trägt dafür Sorge, dass die Blätter (innerhalb der Bearbeitungszeit) sortiert und neu geheftet werden. Das Risiko des Verlustes von einzelnen Blättern trägt der Studierende!
- Es ist nicht erlaubt, Aufgabenstellungen auf gesondertes Papier abzuschreiben.
- Mobiltelefone, Smartphones und Smartwatches sind nicht erlaubt!
- Alle Studierenden bleiben auf ihren Plätzen und verhalten sich ruhig, bis die Klausuren vollständig eingesammelt sind. Folgen Sie den Anweisungen der Aufsichtführenden.
- Bei Täuschung oder versuchter Täuschung wird mit „nicht ausreichend“ bewertet.

Die Klausur besteht aus 6 Aufgaben zu den in der Lehrveranstaltung behandelten Themenbereichen. Die Punktzahl ist bei jeder Aufgabe mit angegeben.

Aufgabe	Mögliche Punkte	Erreichte Punkte
A1	15	
A2	25	
A3	10	
A4	10	
A5	20	
A6	10	
Summe	90	
Evtl. Bonus		

Praxisprojekt in PROG im SS 2024 eingereicht ☐ Nein ☐ Ja

Bitte dokumentieren Sie die Antworten zu allen Aufgaben in der entsprechenden Textbox auf dem Blatt oder der entsprechend benannten Datei (z.B. aufgabe1.txt oder aufgabe5.py) in dem Ordner auf der VM.

Aufgabe 1

- 1) Sie werden gebeten die Architektur von Stud.IP nach dem 4+1 Modell von Kruchten zu beschreiben. Bitte erläutern Sie – jeweils exemplarisch an einzelnen Beispielen – welche Informationen Sie in jeder der 4+1 Sichten zu Stud.IP hinterlegen und dokumentieren würden.

Hinweis: es ist keinesfalls erforderlich, die Architektur von Stud.IP tatsächlich vollumfänglich zu dokumentieren. Es geht lediglich darum, *exemplarisch* zu dokumentierende Inhalte gemäß des 4+1 Modells zu benennen. **7 Punkte**

- 2) Wie beeinflusst das „Magischen Dreieck“ im Software-Engineering die Entscheidungsfindung bei der Projektplanung und welche Strategien können angewendet werden, um die Aspekte des Dreiecks zu balancieren? (4 Punkte)

- 3) Was versteht man unter „Softwarequalität“? Benennen und erläutern Sie vier Beispiele für Qualitätsmerkmale im Software-Engineering. (4 Punkte)

Aufgabe 2

- 1) Was versteht man unter der „Testpyramide“? Bitte benennen Sie die Ebenen der Testpyramide und erläutern diese stichwortartig. [6 Punkte]

- 2) Was versteht man unter Black-Box- sowie White-Box-Tests? Grenzen Sie die Begriffe voneinander ab. [4 Punkte]

3) Gegeben sei folgendes Programm:

```
def foo(x, a):  
    """  
    x: Ein positiver ganzzahliger Parameter  
    a: Ein positiver ganzzahliger Parameter  
    Gibt eine ganze Zahl zurück  
    """  
    zaehler = 0  
  
    while x >= a:  
        zaehler += 1  
        x = x - a  
  
    return zaehler
```

Die Funktion ist mit einem White-Box-Test zu prüfen. Welche Pfade sollten hierfür betrachtet werden? Bitte benennen und erläutern Sie diese kurz. **[3 Punkte]**

Erstellen Sie in der Datei prog_ss24/test/test_aufgabe02.py die korrespondierenden Tests mithilfe von pytest. **[2 Punkte]**

4) Gegeben sei folgendes Programm:

```
def vereinigung(menge1, menge2):  
    """  
    menge1 und menge2 sind Sammlungen von Objekten, von denen  
    jede leer sein könnte. Jede Menge hat innerhalb ihrer  
    selbst keine Duplikate, es kann jedoch Objekte geben,  
    die in beiden Mengen vorkommen. Es wird angenommen, dass  
    die Objekte vom gleichen Typ sind.  
  
    Diese Funktion gibt eine Menge zurück, die alle Elemente  
    aus beiden Eingabemengen enthält, jedoch ohne Duplikate.  
    """  
    if len(menge1) == 0:  
        return menge2  
    elif menge1[0] in menge2:  
        return vereinigung(menge1[1:], menge2)  
    else:  
        return [menge1[0]] + vereinigung(menge1[1:], menge2)  
  
# Beispielaufruf  
print(vereinigung([1, 2, 3], [1, 2]))  
print(vereinigung(["a", "d", "f"], ["f", "t"]))
```

Die Funktion ist mit einem White-Box-Test zu prüfen. Welche Pfade sollten hierfür betrachtet werden? Bitte benennen und erläutern Sie diese kurz. **[6 Punkte]**

Ergänzen Sie in der Datei `prog_ss24/test/test_aufgabe02.py` die korrespondierenden Tests mithilfe von `pytest`. **[4 Punkte]**

Aufgabe 3

Betrachten Sie folgende Funktion `calculate_difference_at_index`, die zwei Listen von Zahlen und einen Index als Parameter entgegennimmt. Die Funktion soll die Differenz zwischen den Zahlen in den Listen am gegebenen Index zurückgeben. **[10 Punkte]**

prog_ss24/src/aufgabe03.py

```
def calculate_difference_at_index(list_c, list_d, idx):  
    """  
    Nimmt an: list_c und list_d sind Listen gleicher Länge mit Zahlen.  
    Gibt zurück: list_c[idx] - list_d[idx]  
    """  
  
    if (  
        not isinstance(list_c, list)  
        or not isinstance(list_d, list)  
        or not isinstance(idx, int)  
    ):  
        raise TypeError  
  
    return float(list_c[idx]) - float(list_d[idx])  
  
# Beispielaufruf und Überprüfung  
assert calculate_difference_at_index([10, 20, 30, 40], [1, 2, 3, 4], 2) == 27  
assert not calculate_difference_at_index([10, 20, 30, 40], [1, 2, 3, 4], 3) == 3
```

Bitte überarbeiten Sie die Funktion `calculate_difference_at_index` um folgendes Verhalten:

- Fängt die Funktion bereits ab, wenn die ersten beiden Parameter keine Listen oder der dritte Parameter kein Integer ist, wird intern eine Exception vom Typ **TypeError** erzeugt. Ändern Sie dies, um eine benutzerdefinierte Fehlermeldung auszugeben.
- Ergänzen Sie die Funktion um die Behandlung von Exceptions vom Typ **IndexError**, wenn der angegebene Index außerhalb des Bereichs der Liste liegt.
- Ergänzen Sie die Funktion um die Behandlung von Exceptions, die auftreten können, wenn nicht-numerische Elemente in den Listen enthalten sind und dadurch die Subtraktion fehlschlägt.
- Nur wenn keine Exception auftritt, geben Sie das Ergebnis zurück.
- Beenden Sie das Programm immer mit einer Nachricht, die bestätigt, dass die Funktion `calculate_difference_at_index` ausgeführt wurde, unabhängig davon, ob eine Exception aufgetreten ist oder nicht.

Aufgabe 4

Gegeben sind die beiden Textdateien bild_1.txt und bild_2.txt, die jeweils mit Sternen gezeichnete Dreiecke zeigen.

Schreiben Sie bitte ein Programm (unter Verwendung von getopt), welches als Argumente auf der Kommandozeile die Namen der beiden Dateien sowie der Ausgabedatei erwartet. Weiterhin soll die Funktion, mit der die beiden Bilder kombiniert werden sollen, angegeben werden. Als Funktion zur Kombination der Dateien kann Logisch-UND sowie Logisch-ODER angegeben werden. Bei Logisch-UND wird im Ausgabebild nur dann ein Stern angezeigt, wenn beide Eingabebilder auch einen Stern erzeugen. Bei Logisch-ODER wird im Ausgabebild nur dann ein Stern angezeigt, wenn in einem der beiden Eingabebilder ein Stern zu sehen ist. **(10 Punkte)**

Die Funktion zum Verknüpfen der Textbilder ist vorgegeben und kann direkt genutzt werden:

prog_ss24/src/aufgabe03.py

```
def combine_images(image1, image2, function):
    combined_image = ""

    if function == "and":
        for i in range(len(image1)):
            if image1[i] == "*" and image2[i] == "*":
                combined_image += "*"
            elif image1[i] == "\n" and image2[i] == "\n":
                combined_image += "\n"
            else:
                combined_image += " "
        elif function == "or":
            for i in range(len(image1)):
                if image1[i] == "*" or image2[i] == "*":
                    combined_image += "*"
                elif image1[i] == "\n" and image2[i] == "\n":
                    combined_image += "\n"
                else:
                    combined_image += " "

    return combined_image + "\n"

# Beispielaufruf
string1 = "***\n * \n  "
string2 = "  \n * \n***"

print(combine_images(string1, string2, "or"))
```

Aufruf:

```
python3 aufgabe04.py --bild1 [name1] --bild2 [name2]
--funktion [and|or] --output [name3]
```

Aufgabe 5

Gegeben ist der folgende unvollständige Code eines Binären Suchbaums.

prog_ss24/src/aufgabe05.py

```
class Node:
    def __init__(self, key):
        self.left = None
        self.right = None
        self.val = key

class BinarySearchTree:
    def __init__(self):
        self.root = None

    def insert(self, key):
        """
        Fügt ein neues Element mit dem gegebenen Schlüssel ein.
        """
        if self.root is None:
            self.root = Node(key)
        else:
            self._insert_rekursiv(self.root, key)

    def _insert_rekursiv(self, node, key):
        pass

    def print_tree(self):
        """
        Gibt die Keys des Baums in aufsteigender Reihenfolge aus
        """
        pass

# Verwendung
bst = BinarySearchTree()
elements = [20, 10, 30, 5, 15, 25, 35]
for elem in elements:
    bst.insert(elem)

bst.print_tree()
```

- a) Bitte implementieren Sie die Funktion `_insert_rekursiv` zum rekursiven Einfügen von neuen Elementen im Binären Suchbaum **(15 Punkte)**
- b) Bitte implementieren Sie die Funktion `print_tree` **(5 Punkte)**

Aufgabe 6

Die Wörter einer Liste sollen nach ihrer Länge aufsteigend sortiert werden.

Gegeben sei hierfür der folgende, leider jedoch unvollständige, Code:

prog_ss24/src/aufgabe06.py

```
def mergesort(input_liste):
    """
    Sortiert die Strings in input_liste aufsteigend
    nach ihrer Länge
    """
    pass

def merge(left, right):
    """
    Führt zwei nach Länge sortierte Listen zusammen und gibt eine
    nach Länge sortierte Liste zurück.
    """
    result = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        if len(left[i]) < len(right[j]):
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result

# Testen Sie Ihre Implementierung mit dem folgenden Code
word_list = ["Hallo", "Python", "Wirtschaftsinformatiker", "Aufgabe", "Sortieren"]

print(mergesort(text))
```

Bitte implementieren Sie die Funktion `mergesort` unter Verwendung des Mergesort-Verfahrens. Zur Vereinfachung ist die `merge`-Funktion schon vorgegeben. (10 Punkte)